

1. JOIN — quando usar?

Regra simples

👉 Use **JOIN** quando:
precisa de dados que estão em tabelas diferentes

Exemplo

Você quer:

- nome do cliente (**clientes**)
- saldo (**contas**)

👉 estão em tabelas diferentes → precisa de JOIN

```
SELECT c.name, a.balance
FROM clientes c
JOIN contas a ON c.cliente_id = a.cliente_id;
```

Tradução

“junte clientes com contas onde os IDs são iguais”

Sinal claro de JOIN

Se a questão fala:

- “nome do cliente E saldo”
- “ator E filme”
- “cidade E país”

👉 é JOIN

2. GROUP BY — quando usar?



Regra simples



Use **GROUP BY** quando:
quer **AGRUPAR** dados pra calcular algo



Exemplo



“quantos filmes cada ator fez”

```
SELECT actor_id, COUNT(*)  
FROM film_actor  
GROUP BY actor_id;
```



Tradução

“agrupa por ator e conta quantos registros ele tem”



Sinal claro de GROUP BY

Se a questão fala:

- “quantos”
- “total”
- “média por...”
- “soma por...”



é **GROUP BY**



JOIN vs GROUP BY (diferença clara)

Situação	Usa
juntar tabelas	JOIN
calcular (COUNT, AVG, SUM)	GROUP BY



Exemplo combinando os dois

👉 “quantos filmes cada ator fez (com nome)”

```
SELECT a.first_name, COUNT(*)  
FROM actor a  
JOIN film_actor fa ON a.actor_id = fa.actor_id  
GROUP BY a.actor_id;
```



Tradução

1. junta ator com filmes
2. agrupa por ator
3. conta



JOIN — explicação clara de verdade



JOIN = juntar tabelas

Mas existem tipos diferentes de “junção”.



1. INNER JOIN (o mais importante)

👉 traz apenas o que tem correspondência nas duas tabelas



Exemplo

```
SELECT c.name, a.balance  
FROM clientes c  
INNER JOIN contas a  
ON c.cliente_id = a.cliente_id;
```



Tradução

👉 “traga só clientes que têm conta”

💡 Visual

client e	cont a
-------------	-----------

João	✓
------	---

Maria	✓
-------	---

Pedro	✗
-------	---

👉 Pedro NÃO aparece

🟡 2. LEFT JOIN

👉 traz tudo da tabela da esquerda

- o que existir da direita
-

🔥 Exemplo

```
SELECT c.name, a.balance
FROM clientes c
LEFT JOIN contas a
ON c.cliente_id = a.cliente_id;
```

🧠 Tradução

👉 “traga TODOS os clientes, mesmo sem conta”

💡 Visual

client e	conta
-------------	-------

João ✓

Maria ✓

Pedro NULL

👉 Pedro aparece com NULL

3. RIGHT JOIN

👉 traz tudo da tabela da direita

Exemplo

```
SELECT c.name, a.balance
FROM clientes c
RIGHT JOIN contas a
ON c.cliente_id = a.cliente_id;
```

Tradução

👉 “traga TODAS as contas, mesmo sem cliente”

Dica de prova

👉 quase ninguém usa RIGHT

👉 pensa assim:

RIGHT JOIN = LEFT JOIN invertido

Resumo simples (GUARDA ISSO)

Tipo	O que traz
------	------------

INNER só o que
combina

LEFT tudo da
esquerda

RIGHT tudo da direita



Como decidir na prova

👉 Se **NÃO** fala nada:

✓ usa **INNER JOIN**

👉 Se fala:

- “todos os clientes mesmo sem conta”
✓ usa **LEFT JOIN**
-

👉 Se fala:

- “mesmo sem correspondência”
✓ provavelmente **LEFT JOIN**
-



Agora testa você (rápido)

Responde:

1.

“listar todos os clientes, mesmo os que não têm conta”

👉 qual JOIN?

2.

“listar apenas clientes que possuem conta”

👉 qual JOIN?

3.

“listar todas as contas, mesmo sem cliente”

👉 qual JOIN?

Responde aí — depois eu te dou exercício prático (bem direto 👍)

o cross é a junção dos left e right

Quase — mas aí você misturou os conceitos 🙄

👉 **CROSS JOIN não tem nada a ver com LEFT/RIGHT**

❌ O que você disse:

“cross é a junção dos left e right”

👉 isso está errado

🟪 O que é CROSS JOIN (de verdade)

👉 **CROSS JOIN = produto cartesiano**

Ou seja:

👉 combina **TODAS as linhas com TODAS as linhas**

🔥 Exemplo

Tabela A:

id

1

2

Tabela B:

letra

A

B

CROSS JOIN:

```
SELECT *  
FROM A  
CROSS JOIN B;
```

Resultado:

id letra

1 A

1 B

2 A

2 B

👉 tudo com tudo

IMPORTANTE (prova)

👉 **CROSS JOIN:**

- quase nunca é usado
 - raramente cai
 - e quando cai é mais teórico
-



Comparação clara

JOIN	O que faz
INNER	só combina iguais
LEFT	tudo da esquerda
RIGHT	tudo da direita
CROSS	tudo com tudo
S	



Dica pra não confundir nunca mais

- LEFT / RIGHT / INNER → usam ON
- CROSS → NÃO usa ON